

Chapter 6

Training and Evaluating Your First CNN

Objective: To train your Convolutional Neural Network (CNN) on the CIFAR-10 dataset, evaluate its performance, and compare it with the fully connected neural network (FCNN) from Chapter 3. You will also explore the outputs of intermediate layers (feature maps) to understand how CNNs learn.

1. Train Your CNN

Training a CNN involves feeding the CIFAR-10 dataset into your model, monitoring the loss and accuracy, and adjusting hyperparameters if needed.

Tasks:

1. Train your CNN using the **model.fit()** function:
 - Use the training dataset (train_images and train_labels).
 - Set a batch size (e.g., 64) and the number of epochs (e.g., 20).
 - Use the validation dataset to monitor the model's performance during training.
2. Plot the training and validation accuracy/loss over epochs. Observe the trends and answer:
 - Is the training accuracy increasing? How about validation accuracy?
 - Is there a point where validation accuracy plateaus or starts decreasing while training accuracy continues increasing? What might this indicate?

2. Evaluate Your CNN

Evaluating the model on the test set will provide an unbiased measure of its performance on unseen data.

Tasks:

1. Use the **model.evaluate()** function to compute the test accuracy and loss.
2. Compare the test accuracy of the CNN to the FCNN you trained in Chapter 3:
 - Which model performs better? Why do you think that is the case?
 - How much higher is the test accuracy of the CNN compared to the FCNN?

3. Analyze Feature Maps

One of the key advantages of CNNs is their ability to learn hierarchical features. In this step, you will visualize the outputs of intermediate layers (feature maps).

Tasks:

1. Use a single image from the test set and pass it through the CNN. Extract the outputs of specific convolutional layers using Keras' Model class.
2. Visualize the feature maps (output of the convolutional layers) for this image:
 - What do the feature maps in the first layer represent? How do they differ from those in deeper layers?
3. Answer:
 - How do the feature maps evolve as the image passes through the CNN? What kinds of patterns or features are being detected?

4. Experiment with Data Augmentation

Data augmentation artificially increases the diversity of the training data by applying transformations like flipping, rotation, and brightness adjustments. This can improve the generalization of your model.

Tasks:

1. Apply data augmentation using TensorFlow's ImageDataGenerator or `tf.keras.layers.RandomFlip`, `RandomRotation`, etc.
2. Retrain your CNN with the augmented dataset and compare its performance to the original CNN.

Programming Assignment:

Follow these steps to complete your programming assignment:

Train the CNN:

1. Use the **`model.fit()`** function with your training and validation sets.
2. Plot the accuracy and loss over epochs.

Evaluate the CNN:

1. Use **`model.evaluate()`** on the test set and print the test accuracy.

Visualize Feature Maps:

1. Create a new model that outputs the intermediate layer activations:

```
from tensorflow.keras.models import Model
```

```
# Define a new model that outputs feature maps from a specific layer
layer_outputs = [layer.output for layer in model.layers if 'conv' in layer.name]
feature_map_model = Model(inputs=model.input, outputs=layer_outputs)
```

2. Pass a single test image through the model and plot the feature maps:

```
import matplotlib.pyplot as plt
```

```

import numpy as np

# Select a test image
test_image = test_images[0:1] # Pick the first image in the test set

# Get feature maps
feature_maps = feature_map_model.predict(test_image)

# Plot the feature maps for the first convolutional layer
for i in range(feature_maps[0].shape[-1]): # Iterate over the filters
    plt.subplot(4, 8, i + 1) # Adjust grid size based on the number of filters
    plt.imshow(feature_maps[0][..., i], cmap='viridis')
    plt.axis('off')
plt.show()

```

Experiment with Data Augmentation:

1. Use TensorFlow's data augmentation layers:

```

from tensorflow.keras.layers import RandomFlip, RandomRotation

# Define the augmentation
data_augmentation = tf.keras.Sequential([
    RandomFlip("horizontal"),
    RandomRotation(0.1),
])

# Apply augmentation to the training set
augmented_train_images = data_augmentation(train_images)

```

2. Retrain the CNN with augmented data and compare its performance to the original CNN.

5. Reflection

Questions to Reflect On:

1. How does the CNN's performance compare to the FCNN? Why do you think CNNs performed better?
2. What patterns did you observe in the feature maps? How do they help the CNN make predictions?

3. Did data augmentation improve the model's performance? Why or why not?